

系统安装手册（前端 + 后端）

适用项目：OHIF Viewer 医学影像平台（含用户认证 auth-api、算法服务 algo-api） 目标：在一台新机器上把整个系统跑起来。前端跑在 Windows，后端服务推荐跑在 Ubuntu 虚拟机的 Docker 里（虚拟机不是强制的，任何能装 Docker 的 Linux 机器都可以；纯本机安装方案见 §4.6）。

0. 获取代码与模型权重

克隆代码（仓库默认分支即开发分支）：

```
git clone https://github.com/falsfe/ohif-viewer.git
```

模型权重文件**不随 git 仓库分发**（单文件超过 GitHub 100MB 限制），需从仓库的 Releases 页面下载：

<https://github.com/falsfe/ohif-viewer/releases>

文件	大小	放置位置（虚拟机）
lung_lobe_model_fp16.pth	约 194M	~/algo-api/logs_Trans/
parenchyma_fp16.pth	约 194M	~/algo-api/logs_parenchyma/

若仓库为私有，需先由仓库所有者在 Settings → Collaborators 中添加你的 GitHub 账号。

1. 系统架构总览

浏览器 (http://localhost:3000)



组件	说明	运行位置	端口
OHIF 前端	React 单页应用 (monorepo)	Windows	3000
auth-api	Node.js + Express + Prisma 认证服务	Windows	4001
algo-api	Python FastAPI 算法服务 (肺叶/肺实质分割等)	Ubuntu VM (Docker)	8000
Orthanc	DICOM 存储服务器 (DICOMweb 插件)	Ubuntu VM (Docker)	8042
MySQL 8.4	用户/研究关联数据库 (库名 ohif_auth)	Ubuntu VM (Docker)	3306

代理配置位于 `platform/app/.webpack/webpack.pwa.js` , 三个后端地址都写死在其中; MySQL 地址写在 `services/auth-api/.env` 。**如果虚拟机 IP 不是 192.168.150.101, 需要同步修改这两处。**

2. 安装清单速查

前端 (Windows)

软件	版本要求	说明
Node.js	≥ 18 (推荐 20 LTS 或 22.x)	仓库 <code>.node-version</code> 为 20.9.0, 当前开发机用 22.20.0 正常
Yarn	1.22.x (Yarn Classic)	仓库 <code>packageManager</code> 锁定 yarn@1.22.22, 不要装 Yarn 2+/Berry
Git	任意近期版本	拉取代码

auth-api (Windows, 与前端共用)

软件	版本要求	说明
Node.js	≥ 18	与前端同一份即可
npm	≥ 6	随 Node 安装

后端 (Ubuntu 虚拟机 / 任意 Linux)

软件	版本要求	说明
Ubuntu Server	24.04 LTS	建议 4 核 / 8G 内存 / 80G 磁盘 (当前 VM 实测占用约 20G)
Docker Engine	任意近期版本 (当前 29.x)	官方 apt 源安装 docker-ce
Docker Compose	v2 插件版 (<code>docker compose</code>)	用于启动 algo-api
MySQL	8.4 (Docker 镜像 <code>mysql:8.4</code>)	容器名 <code>ohif-mysql</code>
Orthanc	Docker 镜像 <code>jodogne/orthanc-plugins:latest</code>	容器名 <code>orthanc</code> , 需 DICOMweb 插件
模型权重文件	—	<code>lung_lobe_model_fp16.pth</code> (约 194M) 、 <code>parenchyma_fp16.pth</code> (约 194M) , 不在 git 仓库里, 从 GitHub Releases 下载 (见 §0)

algo-api 本身的 Python 3.11、PyTorch 等依赖全部打包在 Docker 镜像内 (见 `services/algo-api/Dockerfile`) , 宿主机无需安装 Python。

3. 前端环境安装 (Windows)

3.1 安装 Node.js

从 <https://nodejs.org> 下载 LTS 安装包 (或用 nvm-windows 管理) , 验证:

```
node -v    # 应 >= 18, 如 v22.20.0
```

3.2 安装 Yarn 1 (Classic)

```
npm install -g yarn
yarn --version    # 应为 1.22.x
```

3.3 安装依赖

在仓库根目录 (`viewers/`) 执行:

```
yarn install --frozen-lockfile
```

依赖较大，首次安装需要较长时间。若网络不畅，可先设置国内镜像：

```
yarn config set registry https://registry.npmmirror.com
```

3.4 启动前端

```
cd platform/app  
yarn run dev
```

看到 `[webpack-dev-server] Server started` 即成功，浏览器自动打开 `http://localhost:3000`。

⚠ **注意：**请在 `platform/app` 目录下启动，不要在仓库根目录执行 `yarn dev`（根目录脚本走 `lerna/bun` 流程，本机会报 `bun` 相关错误）。

4. 后端环境安装

后端三个服务（MySQL、Orthanc、algo-api）当前都跑在 Ubuntu 虚拟机的 Docker 里。虚拟机不是强制的——只要是一台能访问的 Linux 机器（物理机、虚机、云主机均可），按同样步骤安装即可。

4.1 虚拟机准备（如使用虚拟机）

1. 安装 Ubuntu Server 24.04 LTS，建议配置：4 核 / 8G 内存 / 80G 磁盘。
2. 配置静态 IP `192.168.150.101`（netplan 配置静态地址，保证重启后 IP 不变）。
3. 配置 SSH 免密登录（在 Windows 上执行）：

```
ssh-keygen -t ed25519          # 如已有密钥可跳过  
ssh-copy-id gin@192.168.150.101  
# 之后直接 ssh gin@192.168.150.101 免密登录
```

4.2 安装 Docker

在 Ubuntu 上按官方源安装 Docker Engine + Compose 插件：

```
# 安装依赖与官方 GPG key
sudo apt-get update
sudo apt-get install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

# 添加 apt 源
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] \
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo $VERSION_CODENAME) stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# 安装
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

# 让当前用户免 sudo 使用 docker（执行后重新登录生效）
sudo usermod -aG docker $USER
```

验证： `docker --version`、 `docker compose version`。

4.3 启动 MySQL 8.4 容器

```
docker run -d --name ohif-mysql \
-p 3306:3306 \
-e MYSQL_ROOT_PASSWORD=StrongRootPass123! \
-e MYSQL_DATABASE=ohif_auth \
-e MYSQL_USER=ohif_user \
-e MYSQL_PASSWORD=StrongUserPass123! \
-v mysql_data:/var/lib/mysql \
--restart unless-stopped \
mysql:8.4
```

生产环境请换成强密码；上面是本地开发环境实际使用的值， `services/auth-api/.env` 中的 `DATABASE_URL` 与之对应。

4.4 启动 Orthanc 容器

先创建配置文件 `~/orthanc-config/orthanc-clean.json`：

```
{
  "Name": "OHIF Orthanc",
  "RemoteAccessAllowed": true,
  "AuthenticationEnabled": false,
  "DicomAet": "ORTHANC",
  "DicomPort": 4242,
  "HttpPort": 8042,
  "StorageDirectory": "/var/lib/orthanc/db",
  "Plugins": [
    "/usr/local/share/orthanc/plugins/libOrthancDicomWeb.so"
  ],
  "DicomWeb": {
    "Enable": true
  },
  "HttpHeaders": {
    "Access-Control-Allow-Origin": "*",
    "Access-Control-Allow-Methods": "GET, POST, PUT, DELETE, OPTIONS",
    "Access-Control-Allow-Headers": "Authorization, Content-Type, Accept"
  }
}
```

然后启动：

```
mkdir -p ~/orthanc-data
docker run -d --name orthanc \
  -p 8042:8042 \
  -v /home/gin/orthanc-data:/var/lib/orthanc/db \
  -v /home/gin/orthanc-config/orthanc-clean.json:/etc/orthanc/orthanc.json \
  --restart unless-stopped \
  jodogne/orthanc-plugins:latest
```

验证：`curl http://192.168.150.101:8042/dicom-web/studies` 返回 `[]` 即正常（空库）。

4.5 部署 algo-api (算法服务)

1. 把仓库里的 `services/algo-api/` 整个目录同步到虚拟机：

```
scp -r Viewers/services/algo-api gin@192.168.150.101:~/algo-api
```

2. 放置模型权重文件（不在 git 中，从 GitHub Releases 下载，见 §0）：

```
~/algo-api/logs_Trans/lung_lobe_model_fp16.pth      # 肺叶分割模型，约 194M
~/algo-api/logs_parenchyma/parenchyma_fp16.pth      # 肺实质分割模型，约 194M
```

3. 构建并启动（`docker-compose.yml` 已在目录内，会自动构建 Python 3.11 + PyTorch CPU 镜像）：

```
cd ~/algo-api
docker compose up -d --build
```

- `compose` 文件里的 `HTTP_PROXY=http://192.168.150.1:7897` 是构建时走宿主机代理加速 `apt/pip` 用的；没有代理可以删掉这两个 `build args`。
- 改了算法代码后，需重新 `docker compose up -d --build` 才生效。
- 镜像包含 PyTorch CPU 版，构建耗时较长，磁盘占用约 2-3G。

4. 验证：

```
curl http://192.168.150.101:8000/api/algo/algorithms
# 返回算法列表 JSON 即成功
```

4.6 替代方案：不用虚拟机 / 不用 Docker (清单)

如需把后端拆开装在物理机或 Windows 上，需要安装：

组件	需要安装的东西
MySQL	MySQL 8.x 社区版（或 MariaDB 兼容版），创建库 <code>ohif_auth</code> 和用户
Orthanc	Orthanc + DICOMweb 插件（Windows 有安装包；或只用 Docker 跑这一个容器），配置同 §4.4
algo-api	Python 3.10–3.11 + <code>venv</code> ；先 <code>pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu</code> ，再 <code>pip install -r services/algo-api/requirements.txt</code> ；启动 <code>uvicorn main:app --host 0.0.0.0 --port 8000</code> ，并设置环境变量 <code>ORTHANC_URL</code> 、 <code>LUNG_MODEL_DIR</code> 、 <code>PARENCHYMA_MODEL_DIR</code> 指向权重目录
auth-api	无额外要求，就是 §5 的 Node 服务，改 <code>.env</code> 中 MySQL 地址即可

Windows 本机跑 Python 时注意：系统 PATH 里的 `python` 可能是 Microsoft Store 占位符，需使用真实解释器路径（当前开发机为 `D:\anaconda3\python.exe`）。

装好后记得把 `platform/app/.webpack/webpack.pwa.js` 里的代理地址和 `services/auth-api/.env` 的 `DATABASE_URL` 改成实际地址。

5. auth-api 安装 (Windows)

```
cd services/auth-api  
npm install
```

配置环境变量 —— 复制 `.env.example` 为 `.env` 并修改（本地开发实际配置如下）：

```
PORT=4001  
NODE_ENV=development  
DATABASE_URL="mysql://ohif_user:StrongUserPass123!@192.168.150.101:3306/ohif_auth"  
JWT_ACCESS_SECRET=local-dev-access-secret-that-is-long-enough  
JWT_REFRESH_SECRET=local-dev-refresh-secret-that-is-long-enough  
JWT_ACCESS_EXPIRES=15m  
JWT_REFRESH_EXPIRES=7d  
COOKIE_DOMAIN=
```

初始化数据库（生成 Prisma Client 并建表）：

```
npm run prisma:generate  
npm run prisma:migrate
```

启动：

```
npm run dev  
# 看到 auth-api listening on http://localhost:4001 即成功
```

6. 启动顺序与验证

每次开发按以下顺序启动：


```
# 1. 虚拟机上的三个容器（VM 重启后需要）
ssh gin@192.168.150.101
sudo systemctl start docker
docker start ohif-mysql orthanc
cd ~/algo-api && docker compose up -d      # algo-api 设了 restart 策略，通常已自动启动

# 2. Windows 终端 A: auth-api
cd viewers/services/auth-api && npm run dev

# 3. Windows 终端 B: 前端
cd viewers/platform/app && yarn run dev
```

验证清单：

检查项	地址	预期
前端	http://localhost:3000	OHIF 界面打开
登录页	http://localhost:3000/auth/login	可注册/登录
auth-api	http://localhost:4001	服务响应
Orthanc	http://192.168.150.101:8042/dicom-web/studies	返回 <code>[]</code> 或研究列表
algo-api	http://192.168.150.101:8000/api/algo/algorithms	返回算法列表 JSON

7. 常见问题

Q: 根目录 `yarn dev` 报 `bun` 相关错误? A: 在 `platform/app` 目录下执行 `yarn run dev` 启动（见 §3.4）。

Q: 虚拟机重启后前端连不上 MySQL / 研究列表为空? A: `sudo systemctl restart docker && docker start ohif-mysql orthanc`，再确认容器 `docker ps` 两个都在运行。

Q: `scp` 到虚拟机报 "No space left on device"? A: 虚拟机磁盘满了，执行 `docker system prune -a -f` 清理无用镜像后重试。

Q: Orthanc 里有数据但登录后看不到研究? A: auth-api 按 `user_studies` 表做用户-研究隔离。通过 curl/Orthanc 界面上传的 study 不会自动绑定用户，需在 MySQL 中手动 INSERT 关联记录（`ohif_auth.user_studies`）。

Q: 医生 PACS 导出的 CT 上传后前端加载崩溃（`undefined[0]`）? A: 导出数据里混入了 Secondary Capture（报告截图等非影像对象），需按 SOPClassUID 过滤删除 SC 再上传。

Q: 想换虚拟机 IP? A: 同步修改两处：`platform/app/.webpack/webpack.pwa.js`（代理目标）和 `services/auth-api/.env`（`DATABASE_URL`）。

8. 附录：关键路径与端口

项目	路径
前端仓库根	viewers/
前端启动目录	viewers/platform/app/
代理配置	viewers/platform/app/.webpack/webpack.pwa.js
auth-api	viewers/services/auth-api/ (.env 配置、 prisma/ 数据库 schema)
algo-api 源码	viewers/services/algo-api/ (main.py 、 algorithms/ 、 nets/ 、 requirements.txt 、 Dockerfile 、 docker-compose.yml)
VM 上 algo-api 部署目录	~/algo-api/ (含 logs_Trans/ 、 logs_parenchyma/ 权重)
VM 上 Orthanc 配置/数据	~/orthanc-config/orthanc-clean.json 、 ~/orthanc-data/

端口	服务
3000	OHIF 前端 (Windows)
4001	auth-api (Windows)
8000	algo-api (VM)
8042	Orthanc HTTP / DICOMweb (VM)
3306	MySQL (VM)
4242	Orthanc DICOM (仅容器内部, 未对外发布)